

```
package com.ndk;
public class fibb {
    public static long fibbJava(int n) {
        if(n == 0 || n==1) {
            return 1;
        }
        else {
            return (fibbJava(n-1)+fibbJava(n-2));
        }
    }
}
//Load the Native code module
static {
    System.loadLibrary("fib");
}
//Prototype for the Native implementation
public static native long fibbNative(int n);
}
```

I have now written a simple Java program (*fibb.java*) that computes Fibonacci numbers recursively. The next step is to generate a C header file based on this Java file; you can do that quickly using Java's standard *javah* tool, with the command *javah -cp src -d jni com.ndk.fibb*



Note: To use *javah*, you need to install the Java JDK.

If you look in the *jni* folder, you will have a header file there, named *com.ndk.fibb.h*, which is generated by the *javah* tool. It will contain standard JNI signatures that are generated using a naming convention that indicates there is a part of the code defined in Java—for example:

```
JNIEXPORT jlong JNICALL Java_com_ndk_fibb_fibbNative
(JNIEnv* env, jclass obj, jint n);
```

The above code tells you that a method called *fibbNative* must be exported, so that it can be called by Java. Notice that the return value is *jlong*, which stands for Java Long, a standard JNI integer value. The variable *env* in the parameter list is a pointer to the virtual machine environment in which the code is executing. The variable *obj* of type *jclass* gives you the class in which this function is defined, while the type *jint* parameter *n* is the argument to the *fibbNative* function.

Let us write the native C version of the above, and call it *fib.c*. Place it in the *jni* folder:

```
#include "com_ndk_fibb.h"
long fibbNative(int n) {
    if (n==0 || n==1)
        return 1;
    else
        return fibbNative(n-1)+fibbNative(n-2);
}
```

```
// Signature as generated by the header file.
JNIEXPORT jlong JNICALL Java_com_ndk_fibb_fibbNative
(JNIEnv* env, jclass obj, jint n) {
    return fibbNative(n); //Signifies that fibbNative should return
the value
}
```

Now that I have written the C version of Fibonacci, let's create an appropriate Makefile for building the required shared library—to build the native code into a module so that Java can call the native function. The Makefile (must be placed in the *jni* directory) should be named *Android.mk*:

```
LOCAL_PATH := $(call my-dir)
include $(CLEAR_VARS)
LOCAL_MODULE := fib
LOCAL_SRC_FILES := fib.c
include $(BUILD_SHARED_LIBRARY)
```

The Makefile is a part of the standard Android *make* system. All it does is take the source file *fib.c* and build it into the module called *fib*. Now that your Makefile is ready, you are ready to build your NDK app. Navigate to the project directory and run the command *ndk-build* (if you added the NDK directory to your system path—else give the absolute path to the extracted NDK's *ndk-build* binary).

If the build is successful, you will now have a *libs* folder with the shared library stored in it. Run a project clean, so that the files generated by *ndk-build* are recognised by your development environment. Now, finally, you need a simple Android application to try out the library we just built—say, with a button that, when pressed, calls the shared module to compute the 15th Fibonacci number:

```
package com.ndk;

import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

public class MainActivity extends Activity implements
OnClickListener{
    TextView t1;
    Button b1;
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        b1=(Button)findViewById(R.id.button1);
    }
}
```